



Fundamentos de Linguagem Python

Do Básico a Aplicações de IA

Paradigmas de Programação

Os paradigmas de programação são modelos ou estilos fundamentais de escrita de código. Eles definem a estrutura, a organização e as regras que os programadores usam para resolver problemas. Não são linguagens, mas sim as filosofias por trás delas. Muitas linguagens modernas, inclusive, suportam múltiplos paradigmas (são multiparadigma), como por exemplo a Linguagem Python.

A seguir estão os **principais paradigmas de programação de computadores**.

1. Programação Imperativa

Este é um dos paradigmas mais antigos e diretos. O foco está em como o programa deve executar uma tarefa. O programador descreve o passo a passo, controlando explicitamente o fluxo de execução e as mudanças de estado (valores de variáveis).

Conceito-chave: Comandos sequenciais que alteram o estado do programa.

Analogia: Uma receita de bolo, onde você segue instruções detalhadas em uma ordem específica: "bata os ovos", "adicione a farinha", "asse por 30 minutos".

Subparadigmas Comuns:

Programação Estruturada: Utiliza estruturas de controle como laços (for, while) e condicionais (if, else) para evitar saltos desordenados no código (o infame GOTO), tornando-o mais legível e organizado.

Linguagens Típicas: C, Pascal, Fortran, Assembly.

2. Programação Declarativa

Em oposição ao imperativo, **o paradigma declarativo foca em o que o programa deve fazer, sem detalhar como fazer. O programador descreve a lógica e o resultado desejado, e a própria linguagem de programação se encarrega de encontrar a melhor maneira de alcançá-lo.**

Conceito-chave: Descrever o resultado desejado, não o processo.

Analogia: Pedir a um artista para pintar "uma paisagem de um pôr do sol na praia", em vez de dar instruções como "pegue o pincel X, use a cor Y no ponto Z da tela".

Subparadigmas Comuns:

- Programação Funcional: (ver abaixo)



Fundamentos de Linguagem Python - Do Básico a Aplicações de IA

- Programação Lógica: Baseada em lógica formal, onde o programa é uma série de fatos e regras. É muito usada em inteligência artificial e sistemas especialistas.

Linguagens Típicas: SQL (para bancos de dados), HTML (para estrutura de páginas web), Prolog (Lógica).

3. Programação Orientada a Objetos (POO)

Este paradigma organiza o código em "objetos", que são instâncias de "classes". Um objeto agrupa dados (atributos) e os comportamentos (métodos) que operam nesses dados. A ideia é modelar o mundo real de forma mais intuitiva.

Conceitos-chave:

- Classe: Um molde ou planta para criar objetos (ex: a classe Carro).
- Objeto: Uma instância de uma classe (ex: meuFusca, um objeto da classe Carro).
- Encapsulamento: Agrupar dados e métodos dentro de um objeto, protegendo os dados de acesso externo indevido.
- Herança: Permitir que uma classe (filha) herde atributos e métodos de outra classe (mãe), promovendo o reuso de código.
- Polimorfismo: Permitir que objetos de diferentes classes respondam à mesma mensagem (chamada de método) de maneiras específicas.

Linguagens Típicas: Java, C++, Python, C#, Ruby, Smalltalk.

4. Programação Funcional (FP)

É um subparadigma do estilo declarativo. Trata a computação como a avaliação de funções matemáticas e evita estados mutáveis e dados compartilhados. As funções são tratadas como cidadãos de primeira classe, o que significa que podem ser passadas como argumentos, retornadas de outras funções e atribuídas a variáveis.

Conceitos-chave:

- Funções Puras: Para uma mesma entrada, sempre produzem a mesma saída e não têm efeitos colaterais observáveis (não alteram variáveis fora de seu escopo).
- Imutabilidade: Os dados não podem ser alterados após sua criação. Em vez de modificar uma estrutura de dados, o programa cria uma nova.



Fundamentos de Linguagem Python - Do Básico a Aplicações de IA

- Composição de Funções: Construir funções complexas combinando funções mais simples.

Linguagens Típicas: Haskell, Lisp, Clojure, F#. Muitas linguagens multiparadigma, como Python, JavaScript e C#, incorporaram fortemente os conceitos funcionais.

Outros Paradigmas Relevantes

Programação Orientada a Eventos: O fluxo do programa é determinado por eventos, como ações do usuário (cliques, teclas pressionadas), mensagens de outros programas ou leituras de sensores. Muito comum em interfaces gráficas (GUIs) e aplicações web.

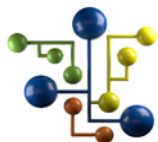
Linguagens: JavaScript (em navegadores), C# (em interfaces Windows Forms ou WPF).

Programação Concorrente: Lida com a execução de múltiplas sequências de instruções ao mesmo tempo. Essencial para sistemas multi-core e aplicações que precisam realizar várias tarefas simultaneamente (ex: baixar um arquivo enquanto a interface do usuário continua responsiva).

Linguagens: Go, Erlang, Rust, Java (com suas bibliotecas de threads).

Resumo Comparativo:

Paradigma	Foco Principal	Exemplo de Linguagem
Imperativo	Como resolver o problema (passo a passo)	C, Assembly
Declarativo	O que precisa ser resolvido (a lógica)	SQL, HTML
Orientado a Objetos	Organização em objetos (dados + comportamento)	Java, C++, Python
Funcional	Uso de funções puras e dados imutáveis	Haskell, Lisp



Equipe DSA

Muito Obrigado!
Continue Trilhando Uma Excelente Jornada de Aprendizagem.